

Project 3: Machine Learning for Predicting Trading Signals

An Executive Breakdown of Technical Indicators, Class Imbalance, and Model Realities

Author: Cesar Juarez

Program: Data Analytics and Artificial Intelligence

Data Handled: 48-Year Historical Equity Data (1970–2018) | 20,860,334 Clean Records

1. Executive Summary: What We Set Out to Do

In stock trading, everyone wants to know when to buy, sell, or sit tight. Rather than guessing, quantitative traders often use mathematical formulas called 'technical indicators' to trigger trading signals automatically. In this project, we set out to answer a clear question: Can standard machine learning models learn to spot these trading opportunities using everyday price behavior, without cheating or peeking at the answers?

We took a massive historical dataset of 20.8 million daily trading records spanning nearly five decades (1970 to 2018) and built two well-known trading indicators: MACD (which watches momentum and trend direction) and RSI (which measures whether an asset has been stretched too far up or down). To protect against bad advice, we set a strict boardroom rule: we only buy or sell when both indicators agree at the exact same time. If they disagree, we do nothing and Hold.

That single rule created an extreme real-world challenge: over 99.9% of all market days were classified as Hold. Actionable Buy and Sell setups occurred less than 0.1% of the time. When we trained our first three machine learning models, every single one fell into the 'Accuracy Paradox.' They proudly boasted a 99.90% accuracy score by simply predicting Hold every single day—meaning they caught zero trades. By re-tuning the algorithms to heavily penalize missed signals, we forced the AI to pay attention to market volatility and short-term price swings, demonstrating how machine learning must be handled when looking for needles in a haystack.

2. Technical Indicators: How We Built the Rules

2.1 MACD: The Momentum Escalator

Think of a stock's price like someone walking on a moving escalator. Daily prices bounce around, but what we really want to know is: Is the escalator moving up or down, and is it speeding up or slowing down? That is what Moving Average Convergence Divergence (MACD) does.

- **12-Day Fast Average:** A moving average of recent prices that reacts quickly to short-term changes.
- **26-Day Slow Average:** A smoother moving average that captures the broader monthly trend.
- **The MACD Line:** The Fast Average minus the Slow Average. When it climbs above zero, short-term buyers are in control.
- **The Signal Line:** A 9-day smoothed average of the MACD line itself, acting as a moving benchmark.

The Rule: A Buy triggers on the exact day the MACD line crosses strictly above the Signal line. A Sell triggers when it crosses strictly below. If no crossover happens today, we stay put (Hold).

2.2 RSI: The Rubber Band Indicator

If MACD tells us the direction of the escalator, the Relative Strength Index (RSI) measures rubber-band stretch. When people buy or sell a stock too aggressively over a two-week period, price gets stretched to extremes and tends to snap back.

- **How it Works:** RSI compares average daily gains against average daily losses over 14 trading days, mapping the result onto a score from 0 to 100.
- **Buy Zone (Oversold < 30):** Relentless selling has stretched the rubber band downward. The stock is deeply discounted and may bounce back.
- **Sell Zone (Overbought > 70):** Aggressive buying has stretched the price too far upward. Momentum is exhausted and due for a pullback.
- **Hold Zone (30 to 70):** Price action sits comfortably within normal historical boundaries.

2.3 The Consensus Rule & The 99.9% Reality Imbalance

Relying on just one indicator often triggers false alarms (whipsaws). To build a disciplined strategy, we required dual confirmation: a Buy fires only when MACD says Buy AND RSI says Buy on the exact same session. A Sell fires only when both say Sell. Any other day is a Hold.

Trading Signal	Total Market Days	Percentage (%)	What is Happening in the Market?
Hold (Class 1)	20,842,154	99.9128%	Normal market conditions; indicators do not agree
Sell (Class 2)	10,103	0.0484%	Overbought price peak paired with downward trend flip
Buy (Class 0)	8,077	0.0387%	Deep oversold discount paired with upward trend flip

3. Data Preparation: Playing by Fair Rules

3.1 The Anti-Cheating Firewall (Target Leakage)

If you give a student an exam with the answers printed on the back, they will score 100% without learning the material. In machine learning, this is called Target Leakage. Because our Buy and Sell signals were created directly using MACD and RSI formulas, we strictly prohibited the models from seeing any MACD or RSI values during training.

Instead, the models were given only everyday market ingredients: percentage price change (daily_return), yesterday's closing price (close_lag_1), rolling moving averages (ma_5, ma_20), 20-day volatility, and the daily high-to-low price spread. The challenge was: Can the AI spot these rare setups using basic price behavior alone?

3.2 No Time Travel (Chronological Splitting)

In standard data science, people often shuffle rows randomly like a deck of cards. In stock trading, that is completely forbidden. If you shuffle time-series data, the model peeks at future market crashes while trying to predict the past. We used a strict chronological split:

- **Training Years (1970 to 2014):** 16.3 million records used exclusively for the models to learn historical patterns.
- **Testing Years (2015 to 2018):** 4.6 million unseen records kept in a locked vault to test how models perform in the future.

To ensure models could be trained smoothly without freezing computer memory, we extracted a clean, stratified representative sample (150,000 training days and 50,000 testing days) that preserved the exact real-world class proportions. Furthermore, our feature scaler was fit strictly on the training partition so no future statistical parameters leaked into the past.

4. The Baseline Experiments & The Accuracy Trap

We trained three distinct machine learning models on our standardized features and evaluated them against the unseen 2015–2018 test data:

Model Name	Accuracy	Buy Recall	Buy Precision	Sell Recall	Sell Precision	Macro F1
Logistic Regression	99.9000%	0.0000	0.0000	0.0000	0.0000	0.3332
Random Forest	99.9000%	0.0000	0.0000	0.0000	0.0000	0.3332
Linear Support Vector Machine	99.9000%	0.0000	0.0000	0.0000	0.0000	0.3332

THE ACCURACY PARADOX REVEALED: All three baseline models achieved a near-perfect 99.90% accuracy, but their Buy and Sell recall was exactly zero! Because 99.9% of days are Hold, the algorithms took a shortcut: guess Hold on every single row. They were right 9,990 times out of 10,000, but completely failed to detect a single trade. In real-world AI, accuracy without context is a vanity metric.

5. Model Improvement: Waking Up the Machine

To break the majority-class shortcut, we chose Random Forest and changed how it evaluates mistakes. By default, an AI treats all mistakes equally. We introduced cost-sensitive class weighting: every time the model missed a rare Buy or Sell, it was penalized roughly 2,000 times harder than if it misclassified a normal Hold day. We also restricted tree depth and leaf sizes to prevent the model from memorizing random noise.

Model Setup	Accuracy	Buy Recall	Sell Recall	Hold F1	Macro F1
Baseline Random Forest (Default)	99.9000%	0.0000	0.0000	0.9995	0.3332
Improved Random Forest (Balanced)	99.8980%	0.0000	0.0000	0.9995	0.3332

The Practical Takeaway: The improved model adjusted its internal split logic and attempted a Buy prediction. However, because our training sample contained only 55 Buys and 70 Sells out of 150,000 rows, individual decision trees often drew zero minority events in their random sub-samples. This reveals a vital lesson in machine learning: when minority events are this sparse (under 0.1%), algorithmic weighting alone isn't enough; you must physically balance the dataset by undersampling the majority class.

6. Feature Importance: What Was the AI Looking At?

By examining how our 150 decision trees made their split choices, we can rank which market features actually helped separate signals from noise:

Feature Name	Importance Score	Share of Influence	What Does This Mean in Plain English?
volatility_20	0.211812	21.18%	Market turbulence: momentum signals happen when volatility expands
daily_return	0.188499	18.85%	Price speed: sharp up or down jumps indicate momentum shifts
daily_range_pct	0.177771	17.78%	Intraday battle: large spreads between daily high and low
ma_20	0.144536	14.45%	Monthly baseline: where is price compared to the 20-day trend?
ma_5	0.141955	14.20%	Weekly baseline: where is price compared to the 5-day trend?
close_lag_1	0.135427	13.54%	Nominal dollar price: the least useful feature for finding setups

Notice the clear pattern: dynamic features (volatility, price speed, and daily range) accounted for nearly 60% of all decision splits. Meanwhile, nominal closing price had the lowest impact. This matches trading intuition: a stock's dollar tag doesn't tell you if it's about to break out; its volatility and momentum do.

7. Strategic Takeaways & Real-World Questions

1. Did the models behave differently across Buy, Hold, and Sell?

Out of the box, all baseline models behaved identically: they predicted Hold 100% of the time. Once we penalized missed signals, Random Forest showed the most flexibility in actively evaluating non-linear volatility interactions.

2. Did high accuracy mean the models performed well?

No. High accuracy was an illusion. A doctor who diagnoses every patient as healthy might be right 99% of the time, but they are useless at catching illnesses. Our models hit 99.9% accuracy while completely failing at their primary job: catching trades.

3. How did class imbalance shape the results?

Because true trading consensus occurred less than 0.1% of the time, standard math algorithms treated genuine trades as meaningless background noise rather than valuable opportunities.

4. What happened when we tackled the imbalance?

Class weighting forced the model to look at minority events. However, because our training slice contained only 55 Buys and 70 Sells out of 150,000 rows, we learned that severe imbalance requires physical resampling (undersampling Holds) alongside cost penalties.

5. Which features were most useful?

Rolling 20-day volatility and daily price return were the clear winners. Breakouts and exhaustion points are driven by velocity and turbulence, not nominal stock prices.

6. What are the drawbacks of using MACD and RSI to define targets?

Both are lagging indicators that look backward. Demanding that both agree at the exact same moment filtered out too many valid setups, creating an extreme imbalance problem that made modeling much harder than it needed to be.

7. What additional features would we add next time?

Trading volume metrics (like On-Balance Volume and VWAP) to track institutional money entering the market, plus macroeconomic indicators (such as the VIX volatility index and interest rates) to understand broader market sentiment.

8. Would these model predictions guarantee profitable trading?

No. Predicting that a technical indicator will cross does not guarantee future profits. In live trading, broker commissions, the spread between bid and ask prices, execution delays, and sudden economic shocks often erase theoretical profits.